Similarities and Differences Between Software Engineers and Software Managers in Agile Teams

Abhinav Pandey
University of Windsor
pandey15@uwindsor.ca

Shrijana Karki
University of Windsor
karki31@uwindsor.ca

Aakriti Mainali
University of Windsor
mainalia@uwindsor.ca

Khemraj Adhikari
University of Windsor
adhika81@uwindsor.ca

Nishan Giri
University of Windsor
giri2@uwindsor.ca

***Abstract*—Software engineers and software managers work on the same product but answer for different things. This study asks whether the two roles in agile teams use the same criteria when they make decisions and judge performance. The team searched OpenAlex and IEEE Xplore by hand. More than 80 papers were scanned and 30 were selected. The team read all 30 in full and used 28. Two industry professionals were also interviewed. The findings were grouped into four themes. The themes cover decisions and communication. They also cover performance and skills and AI tools. The literature describes a clean split where engineers decide how and managers decide what. Practice described joint decisions where engineers frame options and managers choose by business priority. Both roles name the same performance criteria but judge them over different time horizons. The main gap is that manager only samples present collaboration as delegation. The study recommends a shared definition of done that includes maintainability and early risk reporting.**

***Index Terms*—software engineering, software management, agile, performance, decision making, AI tools**

I. Introduction

Most software teams contain two kinds of roles. Software engineers build and maintain the product. Software managers plan the work and answer for budget, deadlines and people. Both roles work on the same product and often sit in the same meetings. Yet they are held to account for different things. This difference matters in industry. When the two roles judge the same work in different ways the team can lose trust and time.

Agile methods were expected to reduce the need for managers. Teams were meant to be self organizing and to govern their own work [1]. In practice the engineering manager still exists in many agile companies. Managers stay for coordination and compliance. They also stay for budget and career growth [2]. Control does not disappear in agile teams. It changes form [3]. Managers still talk to customers and pass priorities to developers [4]. Developers may see a risk while product managers decide its priority and budget [5]. These studies suggest that the two roles share one process but not always one set of goals.

AI tools add a new pressure. Large language models now support coding, testing and documentation. They also support planning and reporting work for managers [6]. Research on these tools serves engineers far more than managers [7]. It is not clear if AI tools will bring the two roles closer or push them apart.

This study asks one research question. In agile software teams do software engineers and software managers use the same criteria when they make decisions and judge performance or do the criteria differ by role? The question is falsifiable. The answer could be that the criteria are the same for both roles.

Before reading the literature the team wrote down an a priori hypothesis. The roles would differ mainly in decision rights and in how performance is judged. They would overlap mostly in skills. This was expected because managers answer to business goals such as budget and deadlines while engineers answer to the quality of the code.

To answer the question this study combines a literature review with two interviews of industry professionals. The findings are grouped into four themes. The first theme is who makes decisions and how the roles talk to each other. The second is how performance is judged. The third is the skills each role needs and the move from engineer to manager. The fourth is the effect of AI tools on both roles.

The rest of the paper is organized as follows. Section 2 describes the literature search and the interviews. Section 3 presents the state of the art from the literature by theme. Section 4 reports the state of practice from the interviews. Section 5 analyses the gaps between theory and practice and tests the hypothesis. Section 6 concludes the paper and gives recommendations and future work.

II. Methodology

This study used two methods. The first was a structured literature search. The second was a set of interviews with industry professionals. The literature gave the theory. The interviews gave a view of current practice.

A. Literature search

The search used two databases. These were OpenAlex and IEEE Xplore. OpenAlex is an open index of scholarly works. IEEE Xplore holds journal and conference papers in engineering and computing. The team ran every search by hand in October 2026. The same search strings were used in both databases. The main keywords were software engineer and software manager. Other keywords were engineering manager and software project manager and technical lead. For the AI group the keywords were large language models and AI agents and generative AI. These were combined with software engineering and team management.

Two filters were applied to every query. Only journal articles and conference proceedings articles were included. This rule

TABLE I
SEARCH STRINGS AND HIT COUNTS IN OPENALEX. THE SAME STRINGS WERE RUN IN IEEE XPLORE

Group	Exact phrase	Other terms	Hits
2000 to 2019	software engineer plus software manager	none	95
2000 to 2019	engineering manager	software	778
2000 to 2019	software project manager	role	283
2000 to 2019	technical lead	software team	353
2020 onward	software engineer plus software manager	none	58
2020 onward	engineering manager	software	1057
2020 onward	software engineering management	none	247
2020 onward	technical lead	software team	545
2022 onward	large language models	software engineering roles	36257
2022 onward	AI agents	software development team management	4317
2022 onward	generative AI	software engineers managers	4156

was used to keep the focus on formally published work. Queries were split into three groups by publication date. The first group covered 2000 to 2019. The second group covered 2020 onward. The third group covered 2022 onward and targeted LLM and agentic AI work. The split was used so that older and newer views of the roles were both captured. Exact phrase matching was used for the terms in the exact phrase column of Table 1. In the first and fifth rows both phrases were required.

The hit counts were too large to read in full. The team looked at the top ranked results of each query instead. After duplicates were removed 137 records were kept for a first look.

The team then scanned more than 80 of these papers by hand. Each paper was checked by its title and abstract. The team used simple rules. The paper had to be a journal article or a conference paper so that it was formally published. It had to be about software so that it fit the field. It had to mention a management or leadership role so that it fit the research question. The year was also checked so that older and newer work were both included. Papers outside the domain were dropped. Papers with no abstract or no clear venue were also dropped.

From the scanned papers 30 were selected for full reading. The team aimed for about 15 papers before 2020 and about 10 from 2020 onward. About 5 more were on LLMs and AI agents. Open access was required so that every member could read the full text. The papers were split among five team members with six papers each. Each member read every assigned paper in full. Each member wrote a summary with a fixed template. The template recorded the title and the type of study and who was studied. It also recorded the main finding and the engineer and manager contrast. It ended with one quote

TABLE II
NUMBER OF PAPERS AT EACH STEP

Step	Papers
Records kept for a first look	137
Scanned by hand	80+
Assigned and read in full	30
Used in the final paper	28

or number and a keep or drop decision. In the end 28 papers were used in this paper. One was excluded as off topic with no role content. One replacement paper had not yet been read. A small number of sources are preprints or conceptual papers with no data. These are used lightly and flagged in the text.

B. Interviews

Two industry professionals were interviewed. They are named only as Professional A and Professional B. Both hold a B.Eng degree. Professional A is a software engineer with more than 3 years of industry experience. Professional B is a senior software engineer with more than 5 years of experience. Professional A was chosen because the daily work of this person is building software inside an agile team. This gives a direct engineer view of decisions and performance. Professional B was chosen because the senior level brings more contact with leads and managers. This gives a view closer to leadership. Neither is a friend or family member of the team.

The interview with Professional A was spoken. A team member asked four open questions. The interview was recorded and then transcribed. Professional B gave written answers to three open questions. The questions covered how decisions are made and how performance is judged. They also covered how the roles communicate and the effect of AI tools.

Consent was obtained from each professional before recording or before written answers were collected. The work was run as a student project under the TCPS 2 rules and the university REB exemption. The questions asked about general industry practice. They did not ask about the company of the interviewee. No names or company names appear in this paper. The anonymized transcripts are provided as appendices.

III. STATE OF THE ART

This section reports what the reviewed literature says about the two roles. The 28 sources are grouped into the four themes of this study. For each theme the section states what is similar between software engineers and software managers and what is different. Most sources come from agile or project settings. Several are weak or conceptual and are used only as support.

A. Who makes decisions and how the roles talk to each other

The literature agrees that agile did not remove the manager. Control changes its form but does not disappear [2], [3]. A review of 76 papers explains that managers persist for coordination and compliance and also for budget and careers [2]. The same review links engineering managers in three banks to 25 percent fewer integration defects [2]. Many studies

describe a similar split of decision rights. Engineers decide how and when the technical work is done. Managers decide what is built and why and with what budget [3], [5], [8], [9]. In crowdsourcing contests managers decide what and when to crowdsource and how to do it. Engineers solve the tasks [9]. In security work developers see the risk but product managers set the priority and the budget [5]. Developers then must sell security in business words. Short workshops improved product manager engagement in 6 of 8 groups [5].

Communication follows the same line. Team leads take customer calls and pass priorities down in weekly meetings, which protects developers from outside pressure [4]. Junior developers depend on managers for the big picture [4]. Managers must also clarify business objectives, and engineers depend on that clarity [10]. In one survey of 100 project managers the item that employees know the objectives ranked lowest at 62.9 percent [10].

Not all sources draw the line so sharply. Agile teams in 19 companies governed themselves through their own coding and architecture standards [1]. Of 254 artefact uses, 94 were driven by the agile method itself [1]. A review proposes that the manager moves from director to coach to consultant as the team matures [2]. These sources suggest that the roles start apart and can move closer. Decision rights can also hold engineers back. Developers in one survey found the cloud suitable. Yet management was less likely to migrate. Sunk cost may explain this [11]. Top management support was the strongest driver of actual use, with a weight of 0.636 [11]. The similarity is that both roles value open communication and shared goals [12], [13]. The difference is that managers hold priority and budget, while engineers hold the technical choices.

B. How performance is judged

Here the roles share some criteria but weigh them differently. Managers tend to judge by delivery against time and budget. In one study 11 of 12 managers defined productivity as finishing on time without sacrificing quality [14]. Those managers found status reports and check ins slow and biased and wanted an objective tool [14]. Developers were never asked in that study. Several manager studies link success to customer engagement and product owner value and sponsor funding [12], [15]. The same managers also valued engineering quality, as 90 percent said good engineering practices are key [15]. So quality appears in both views. Estimation is a weak point on the manager side. Managers in Jordan repeated estimation mistakes and overran time and budget [16]. Usability was affected by 59.70 percent when the cost estimate failed [16]. In Iran 83 percent of managers assessed risk only implicitly and 93 percent never used a risk tool [17]. The risks they listed were mostly about staff, such as reliance on a few skilled developers and turnover [17].

A modelling study shows how engineer behaviour feeds manager decisions. Tester learning and debugger negligence shape the release time and cost. The example release came at 3.45 months for 125441 dollars [18]. Training improves testers but costs money, so the manager faces a trade off [18]. Engineers also judge managers. Engineers rated project managers on approachability and fairness and decision making. Skills and training mattered more than gender [19]. In that sample 82.5 percent of male and 50 percent of female employees chose male managers as more effective [19]. Engineers in agile teams link their happiness to present leaders and effective communication [13]. A conceptual paper warns that metrics used to punish people destroy trust [3]. A second conceptual paper claims that product managers push for speed and engineering managers push for quality [20]. That claim has no data and serves only as support. The similarity is that both roles count quality and delivery. The difference is that managers measure outcomes on schedule, while engineers stress code quality and how managers treat the team.

C. Skills each role needs and moving from engineer to manager

The skill theme shows the largest overlap. A study of quality assurance engineers found 25 attributes in five groups [21]. Of these 21 of 25 or 84 percent also appeared in studies of software engineers [21]. Curiosity was named by 20 of 25 interviewees [21]. Autonomy was the one attribute mentioned only for managers [21]. Several studies agree that communication and collaboration are core to both roles [6], [12], [13]. In one study 90 percent of managers called collaboration and communication core success factors [12]. Engineers agree that soft skills are needed, yet they take them for granted because technical skills dominate [22]. Those developers asked for courses and workshops [22].

The difference lies in which skills each role uses most. Managers need technical skill and people skill at the same time [2]. Managers also prefer servant leadership, and 9 of 10 chose it [8]. This uniform result is unusual and is treated with caution. A study of data intensive projects found that each role saw the same problems from its own position [23]. Executives spoke about governance, engineers about hands on tools and team leads about culture and communication [23]. A development manager said that software engineering is his craft and so he does not compromise much [23]. A chief information officer said he worried less about technical detail once the right people were in place [23]. This suggests that the move into management shifts attention from code to people. The literature says little about how engineers make that move. No reviewed study followed engineers as they became managers.

D. The effect of AI tools on both roles

The AI literature shows a clear imbalance. A survey of 988 studies found that only 7 of 926 LLM studies address software management tasks [7]. AI research therefore serves engineers far more than managers. For engineers AI tools support code completion and tests. They also support documentation and review [24], [25]. Humans still need to check the output, and the engineer shifts toward checking and guiding the AI [25]. Scriptless testing may lower the scripting skill needed, so that

non technical people can test [26]. For managers, generative AI acts as an assistant or friend rather than a replacement [6]. Managers use it for communication, repetitive tasks and prediction [6]. AI tools also take over repetitive tasks across all Scrum roles [27]. About 20 percent of teams had started using AI tools and 43 percent planned to [27]. A conceptual paper splits AI uses into engineer tasks and manager tasks [28]. It notes that the same prompt gives different answers, so people must check the results [28]. Developer fear of job loss is also reported [24]. The similarity is that both roles use AI for routine work and must still verify it [6], [25], [28]. The difference is in the tasks. Engineers use AI for code and tests, while managers use it for reports and planning. Several of these AI sources are preprints or short reviews or conceptual papers [24]–[26], [28]. They are used only as support.

The evidence base behind these themes is uneven. Only a few studies asked both roles [1], [4], [5], [11], [13], [23]. Many studies asked only managers [8], [10], [12], [14]–[17]. Some asked only engineers or quality staff [19], [21], [22]. Other sources are conceptual or experience based. Some are models with no human participants [3], [18], [20], [27], [28]. Several are reviews of earlier work [2], [6], [7]. The weaker sources are used only as support. One key reference is a Microsoft study of what makes a great software manager. At least two reviewed papers point to it [20], [21]. This study has not yet read that work, so it is not cited here.

IV. State of practice

This section reports what the two professionals said. It follows the four themes. Professional A gave a spoken interview with four open questions. Professional B gave short written answers to three questions. The answers of Professional B are brief and carry less detail. Some themes therefore rest on one voice only. This is noted under each theme.

A. Who makes decisions and how the roles talk to each other

Both professionals spoke to this theme. Professional A said that the common picture of engineers deciding how and managers clearing blockers is generally the idea. In reality the process is more collaborative. Engineers understand the technical details best. They are in the best position to judge implementation approaches and technical feasibility. They also judge effort and estimates. Professional A gave an example. A product manager explains a feature that customers need. The engineering team finds two ways to build it. One way takes about two weeks. Another delivers most of the value in about three days. The engineers explain the tradeoffs. The product manager helps choose based on business priorities. A team lead bridges the two views. Professional A said that engineers should have autonomy over technical decisions. The best decisions happen when engineers and managers work together.

Professional B said that most of the time decisions are team decisions. Work is done through collaboration and joint discussion. Professional B also said that leads and managers still talk to customers. They pass priorities to developers during sprint planning. In this account customer contact still runs through the lead or the manager.

B. How performance is judged

Only Professional A spoke to this theme. Professional A said that it depends on the organization and the manager. In general performance is judged on delivery and quality of work. Problem solving and communication and collaboration also count. Managers may look at whether someone delivers reliably, contributes to team goals and communicates risk early. Engineers may look more closely at technical quality and maintainability. Professional A gave an example. A developer may finish a feature quickly. If the work creates technical debt or recurring bugs it may not be a good sign of overall performance. Someone who takes longer because they found an unexpected technical issue and solved it properly may create more long term value. Professional A said that both views are important. Good performance is not only finishing tasks on time. It is delivering useful and reliable software and working well with the team.

Professional A also described how progress is tracked. It comes down to trust, clear expectations and good communication. At the start of a task the team should understand the requirements, the expected outcome and the estimated effort. After that managers should give engineers space to work out the implementation. Regular standups and progress updates help find blockers early. Constant requests for updates or checks of every small detail interrupt the work. They can make the developer feel that they arent trusted. Professional A said that estimates arent always accurate and that unexpected problems appear during the work. In that case Professional A would rather see an engineer raise the issue early and discuss a revised estimate. The alternative is pressure to deliver something unrealistic. Managers should track outcomes, risks and blockers rather than control every step.

C. Skills each role needs and moving from engineer to manager

Only Professional B spoke directly to this theme, and briefly. Professional B said that a good engineer may not naturally make a good manager. The engineer may not have leadership experience or skills. If engineers have or develop leadership skills they can become good managers. Professional B did not say which skills matter most or how they are built. Professional A did not address the move into management. Professional A did say that a team lead must understand team capabilities and support technical choices. This suggests that the lead role mixes technical and people skills.

D. The effect of AI tools on both roles

Only Professional A spoke to this theme. Professional A said that AI will change both roles significantly but in different ways. For engineers AI can help write routine code and generate tests. It can also explain unfamiliar code and suggest ways to debug. Building reliable software involves more than generating code. Engineers still need to understand

the system and validate AI generated solutions. They still make architecture decisions, consider security and understand the impact on users. For managers AI can help summarize meetings and prepare reports. It can also organize information and support planning and estimation. Prioritizing work still needs an understanding of customers and business strategy. It also needs an understanding of team capabilities and organizational context. Professional A expected AI to automate some repetitive tasks in both roles. It would not fully replace either role. Engineers may spend less time on routine code. Managers may spend less time gathering information and preparing reports.

V. Gap analysis and discussion

This section compares the literature with the interviews theme by theme. It looks for the root cause of each gap and the evidence for that cause. It then states what the gap costs and what it means for the hypothesis. Neither side is treated as the correct one. Each view is shaped by who produced it and under which constraints.

A. Who makes decisions and how the roles talk to each other

The literature mostly describes a clean split. Engineers decide how and when the work is done and managers remove blockers [2], [8]. Managers set goals and guardrails while engineers work freely inside them [3]. Managers decide priority and budget and what to outsource [5], [9]. In practice both professionals described joint decisions. Professional A described a specific pattern. Engineers frame options with effort and value attached. The product manager then chooses between them by business priority. This is neither pure delegation nor pure control. Neither the servant leadership view [8] nor the control view [3] models this exchange of options.

The likely root cause is who was sampled. The study that states the clean split most directly asked only managers [8]. Several other sources in the evidence base are also manager only [10], [12], [14], [17]. A manager describing their own role may present collaboration as delegation. The evidence for this cause is that studies sampling both roles show more exchange. Developers had to sell security in business words before product managers acted [5]. Leads passed priorities down in weekly meetings and juniors depended on them for the big picture [4]. Each role saw the same problems from its own position [23]. The uniform results in the main manager only study also call for caution [8].

The consequences appear when the exchange fails. Security work can be skipped when engineers cannot translate risk into priority [5]. Managers may also hold on to past investments that engineers no longer find suitable [11]. The hypothesis is partly confirmed. Decision rights do differ, but by type of input and not by decision. Engineers own feasibility and effort. Managers own priority. The decision itself is shared. Theory has missed this options based joint decision as a distinct way of working.

B. How performance is judged

The literature offers a manager definition of productivity. Most managers defined it as on time completion without sacrificing quality [14]. Managers also found status reports and check ins slow and biased and welcomed a monitoring tool [14]. Practice shows a similar list of criteria. Professional A named delivery and quality. Problem solving and communication and collaboration were also named. The difference lies in weight and time horizon. Managers in the account of Professional A look at reliable delivery now and early risk reporting. Engineers look at maintainability and the prevention of future issues. One view looks at this sprint. The other looks at the next year of the code.

The root cause is accountability. Managers answer to deadlines and budgets, and estimates often overrun both [16]. Engineers live with the code after release. Evidence for this cause comes from several directions. Professional A gave the example of a fast feature that adds technical debt. A development manager who saw engineering as a craft resisted compromise [23]. A release model also shows that more training improves quality but costs money, which forces a trade off [18]. A similar split between speed and quality is proposed for product and engineering managers, though without data [20].

The consequences are technical debt, recurring bugs and loss of trust. Metrics used to punish people destroy trust [3]. Professional A said that constant update requests signal distrust. All 12 managers liked a screenshot based tool, yet developers were never asked [14]. Communication and present leaders are linked to happiness in agile teams [13]. Heavy monitoring may therefore carry a risk to wellbeing, though no source here measures burnout. The hypothesis is partly confirmed. The criteria share names but differ in horizon. Theory has missed that honest early risk reporting is itself a criterion both roles value. Estimates are unstable in practice. A tool that measures activity cannot see a revised estimate raised early.

C. Skills each role needs and moving from engineer to manager

Here theory and practice largely agree. Great quality assurance engineers share 21 of 25 attributes with software engineers, about 84 percent [21]. Autonomy appeared only for managers [21]. Managers need both technical and people skills [2]. Engineers judge managers on approachability and fairness and decision making. Skills and training matter more than gender [19]. Professional B said that a good engineer may not make a good manager without leadership skills. The gap is in focus rather than content. The literature lists attributes at one point in time. It rarely follows the move from engineer to manager.

The root cause is research design. Attribute studies are cross sectional and often sample one role [21], [22]. Soft skills are also taken for granted because technical skills dominate [22]. This may explain why the gap between technical and leadership skill is noticed only at promotion. The consequences

are cost and team strain. Teams already rely on a few skilled developers [17]. Promoting one of them without leadership training may affect both delivery and team climate [13]. The hypothesis is confirmed on overlap. Practice adds that overlap is not enough. Leadership is the skill that separates the roles. This finding rests on one short written answer and is weak.

D. The effect of AI tools on both roles

The research base is uneven. Only 7 of 926 LLM studies address software management tasks [7]. Most work shows the engineer shifting toward checking and guiding AI output [25]. Evidence on managers comes mainly from grey literature and conceptual work [6], [28]. Practice shows a more balanced expectation. Professional A expected both roles to change. Engineers keep validation, architecture and security. Managers keep prioritization and business context. Neither role is replaced. This matches the assistant view of managers using GenAI [6].

The root cause is likely tooling and measurability. Coding tasks have clear outputs that are easy to test. Management tasks such as prioritization are harder to measure. The evidence is the imbalance in the survey itself [7]. Adoption is also still early, with about 20 percent started and 43 percent planning [27]. The consequences include security risk from unchecked output, since the same prompt can give different answers [28]. Managers may also adopt tools with little evidence behind them. Fear of job loss among developers is also reported [24]. AI was not part of the hypothesis. The findings suggest that AI may move both roles toward judgement and checking. That would increase the overlap. Theory has missed that the change may be symmetric across roles.

E. Overall verdict

The hypothesis is partly confirmed. The roles do overlap mostly in skills. They do differ in decision rights and in how performance is judged. The difference is smaller than expected. It lies in which input each role brings and in the time horizon of each judgement. The because clause holds only in part. Managers in the evidence also value quality [14]. Professional A is an engineer and still valued delivery and early risk reporting. The line between business goals and code quality is not clean in practice.

F. Limitations

This study has clear limits. Only two professionals were interviewed. Both are engineers and no manager was interviewed. So the manager view in practice is missing. Professional B gave short written answers. Some themes therefore rest on one voice. Papers were selected only if they were open access. This is a selection bias because paywalled work may say something different. Several sources come from weak venues. Some are conceptual and have no data. Many manager studies asked only managers. The search did not cover ACM or other databases. The findings should be read as indicative and not general.

VI. Conclusion and future work

This study asked whether software engineers and software managers in agile teams use the same criteria to make decisions and judge performance. The literature and two interviews were compared across four themes. The answer is mixed. The roles share most skills and most criteria by name. They differ in the input each brings to a decision and in the time horizon of their judgements. Decisions in practice were described as joint and options based rather than delegated. Performance was described as delivery now for managers and maintainability later for engineers. AI tools were expected to change both roles, while research so far serves engineers far more [7].

Two recommendations follow for industry. First teams should agree on a shared definition of done. It should include maintainability and early risk reporting next to on time delivery. Both engineers and managers should agree on it together. This makes the difference in time horizon visible and negotiable. Second managers should track outcomes and risks and blockers rather than activity. Activity measures such as screenshots can damage trust [3], [14]. A revised estimate raised early should count as good performance and not as failure. Standups can then focus on blockers and risks. Constant status checks become less needed.

Theory should update in three ways. Current models tend to assume that each role holds its own criteria and its own decisions. Models of agile leadership should include the options based joint decision as its own pattern. Studies of performance should ask both roles the same questions. Research on AI in software work should give more attention to management tasks. It should also study how checking AI output changes the work of both roles.

Future work should first extend the literature search to ACM and to paywalled papers. More interviews are needed with a balance of engineers and managers. A useful study would ask matched pairs of an engineer and their manager the same performance questions. This would show directly whether the criteria differ by role. It would also reduce the bias of manager only samples. A second study could follow engineers through their first year as managers to see which skills they lack and how they build them.

References

[1] G. Wagenaar, S. Overbeek, G. Lucassen, S. Brinkkemper and K. Schneider, Working software over comprehensive documentation, rationales of agile teams for artefacts usage, Journal of Software Engineering Research and Development, Springer, 2018, doi: 10.1186/s40411-018-0051-7.

[2] R. Kalluri, Why does the engineering manager still exist in agile software development, International Journal of Software Engineering and Applications, vol. 16, no. 4/5, 2025, doi: 10.5121/ijsea.2025.16502.

[3] D. C. Kurt, Agility without chaos, managerial control mechanisms in agile software development organizations, IRE Journals, vol. 8, no. 1, 2024, doi: 10.64388/irev8i1-1714635.

[4] E. Knauss, A. Yussuf, K. Blincoe, D. Damian and A. Knauss, Continuous clarification and emergent requirements flows in open commercial software ecosystems, Requirements Engineering, Springer, 2018, doi: 10.1007/s00766-016-0259-1.

[5] C. Weir, I. Becker and L. Blair, Incorporating software security using developer workshops to engage product managers, Empirical Software Engineering, Springer, 2023, doi: 10.1007/s10664-022-10252-0.

[6] L. I. Assalaarachchi, Z. Masood, R. Hoda and J. Grundy, Generative AI for software project management, insights from a review of software practitioner literature, IEEE Software, 2025, doi: 10.1109/ms.2025.3619936.

[7] Q. Zhang, C. Fang, Y. Xie, Y. Zhang, S. Yu, W. Sun, Y. Yun and Z. Chen, A survey on large language models for software engineering, Science China Information Sciences, vol. 69, no. 4, 2026, doi: 10.1007/s11432-025-4670-0.

[8] D. S. Nguyen, Workplace factors that shape agile software development team project success, American Scientific Research Journal for Engineering, Technology, and Sciences, vol. 17, no. 1, pp. 323–391, 2016.

[9] S. Bibi, I. Zozas, A. Ampatzoglou, P. G. Sarigiannidis, G. Kalampokis and I. Stamelos, Crowdsourcing in software development, empirical support for configuring contests, IEEE Access, vol. 8, p. 58094, 2020, doi: 10.1109/access.2020.2982619.

[10] M. M. Tarawneh, The alignment between business objectives clarity and software objectives, Computer Engineering and Intelligent Systems, vol. 10, no. 2, pp. 28–32, 2019, doi: 10.7176/ceis/10-2-04.

[11] E. Pisirir, O. Chouseinoglou, C. Sevgi and E. Ucar, Factors affecting the adoption of cloud for software development, a case from Turkey, International Journal of Information Technology and Decision Making, 2023, doi: 10.1142/S0219622023500517.

[12] D. S. Nguyen, Success factors as critical that shape agile software development project success, International Journal of Sciences Basic and Applied Research, vol. 25, no. 3, pp. 20–69, 2016.

[13] L. F. Amorim, M. Marinho and S. Sampaio, How (un)happiness impacts on software engineers in agile teams, International Journal of Software Engineering and Applications, vol. 11, no. 3, 2020, doi: 10.5121/ijsea.2020.11303.

[14] Suwarno, Y. Christian, K. Yoputra and Y. Estrada, Exploring machine learning to support software managers insights on software developer productivity, Journal of Applied Data Sciences, vol. 6, no. 3, pp. 1504–1524, 2025, doi: 10.47738/jads.v6i3.661.

[15] D. S. Nguyen, Success factors that influence agile software development project success, American Scientific Research Journal for Engineering, Technology, and Sciences, vol. 17, no. 1, pp. 172–222, 2016.

[16] M. M. Tarawneh and S. R. Allahawiah, Reliable estimate and software development, Jordanian environment, International Journal of Computer Applications, vol. 179, no. 44, pp. 10–14, 2018, doi: 10.5120/ijca2018917068.

[17] A. R. Afshari, V. Brtka and M. Cockalo Hronjec, Project risk management in Iranian software projects, Journal of Engineering Management and Competitiveness, vol. 8, no. 2, pp. 81–88, 2018, doi: 10.5937/jemc1802081a.

[18] C. W. Yeh and C. C. Fang, Software testing and release decision at different statistical confidence levels with consideration of debuggers learning and negligent factors, International Journal of Industrial Engineering Computations, vol. 15, pp. 105–126, 2024, doi: 10.5267/j.ijiec.2023.11.001.

[19] A. A. Qureshi, S. Afzal, I. Daud and M. A. Saleem, A comparative analysis of gender based management styles of software project managers, International Journal of Computer Applications, vol. 71, no. 14, 2013, doi: 10.5120/12425-2260.

[20] A. Babu, Parental leadership in product engineering partnerships for managing software teams, International Journal on Science and Technology, vol. 16, no. 2, 2025, doi: 10.71097/ijsat.v16.i2.6225.

[21] R. S. Farias, I. Ahmed and E. S. de Almeida, What makes a great software quality assurance engineer, IEEE Transactions on Software Engineering, vol. 51, no. 4, 2025, doi: 10.1109/tse.2025.3542763.

[22] T. Zoric and Z. Stojanov, Software developers perceptions of soft skills in software requirements engineering, Journal of Engineering Management and Competitiveness, vol. 8, no. 1, pp. 54–64, 2018, doi: 10.5937/jemc1801054z.

[23] U. M. Graetsch, H. Khalajzadeh, M. Shahin, R. Hoda and J. Grundy, Dealing with data challenges when delivering data intensive software solutions, IEEE Transactions on Software Engineering, 2023, doi: 10.1109/tse.2023.3291003.

[24] X. Zhao, T. Sun, S. Ren, J. Yang and Y. Liu, RAG based AI agents for enterprise software development, implementation patterns and production deployment, Frontiers in Artificial Intelligence Research, vol. 2, no. 3, pp. 501–520, 2025, doi: 10.71465/fair456.

[25] C. Gao, X. Hu, S. Gao, X. Xia and Z. Jin, The current challenges of software engineering in the era of large language models, ACM, 2025, doi: 10.1145/3712005.

[26] G. Viswanathan, AI agentic scriptless automation in software testing, International Journal of Computer Trends and Technology, vol. 72, no. 9, pp. 120–125, 2024, doi: 10.14445/22312803/ijctt-v72i9p118.

[27] D. Pashchenko, Excellence practices in the Scrum paradigm in software development, Journal of Data Science and Intelligent Systems, vol. 3, no. 2, pp. 79–86, 2025.

[28] A. Bahi, J. Gharib and Y. Gahi, Integrating generative AI for advancing agile software development and mitigating project management challenges, International Journal of Advanced Computer Science and Applications, vol. 15, no. 3, pp. 54–61, 2024, doi: 10.14569/ijacsa.2024.0150306.